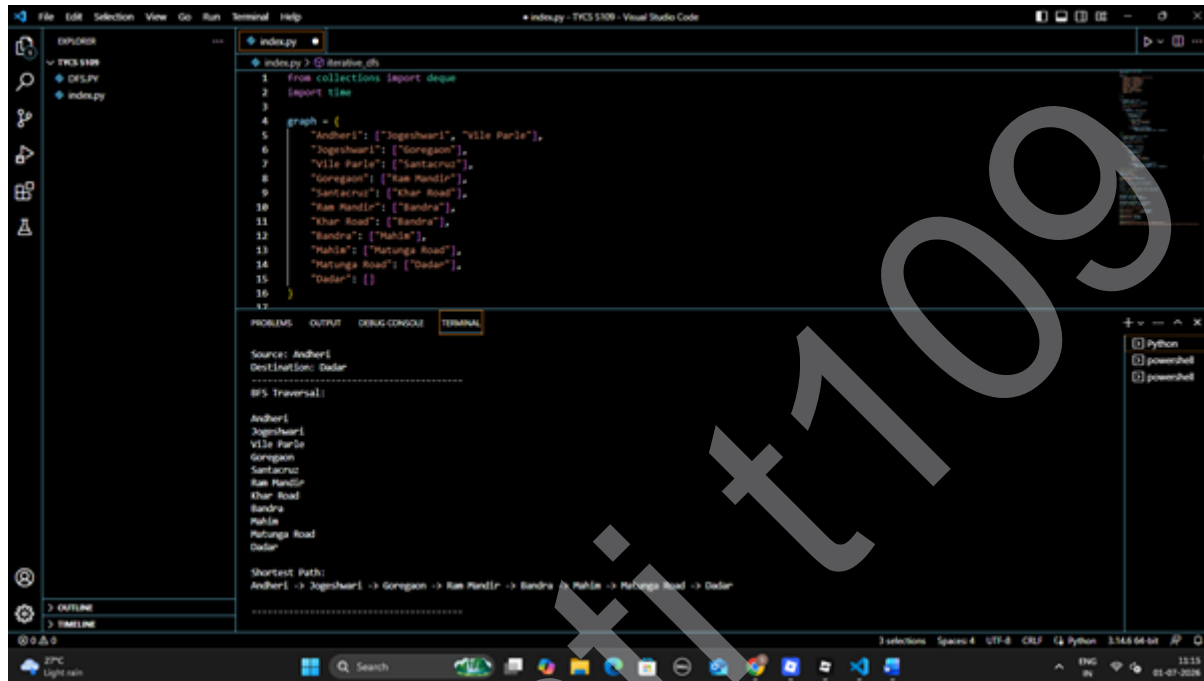


SHETH L.U.J AND SIR M.V COLLEGE
PRACTICAL NO 1

- AIM - A) Implement the Breadth First Search algorithm to solve a given problem.
B) Implement the Iterative Depth First Search algorithm to solve the same problem.
C) Compare the performance and efficiency of both algorithms.

A.



The screenshot shows a Visual Studio Code editor with a Python file named `index.py`. The script defines a graph with nodes and edges, and implements a BFS algorithm to find the shortest path from 'Andheri' to 'Dadar'.

```
1 from collections import deque
2 import time
3
4 graph = {
5     "Andheri": ["Jogeshwari", "Vile Parle"],
6     "Jogeshwari": ["Goregaon"],
7     "Vile Parle": ["Santacruz"],
8     "Goregaon": ["Kam Mandir"],
9     "Santacruz": ["Khar Road"],
10    "Kam Mandir": ["Bandra"],
11    "Khar Road": ["Bandra"],
12    "Bandra": ["Mahla"],
13    "Mahla": ["Matunga Road"],
14    "Matunga Road": ["Dadar"],
15    "Dadar": []
16 }
17
```

The terminal output shows the BFS traversal and the shortest path:

```
Source: Andheri
Destination: Dadar
-----
BFS Traversal:
Andheri
Jogeshwari
Vile Parle
Goregaon
Santacruz
Kam Mandir
Khar Road
Bandra
Mahla
Matunga Road
Dadar

Shortest Path:
Andheri -> Jogeshwari -> Goregaon -> Kam Mandir -> Bandra -> Mahla -> Matunga Road -> Dadar
```

B

NAME – UNNATI RATHOD
ROLL NO – T109
SUBJECT – AI

SHETH L.U.J AND SIR M.V COLLEGE
PRACTICAL NO 1

```
indexpy 2 @ Relative.dh
1 from collections import deque
2 import time
3
4 graph = {
5     "Andheri": ["Jogeshwari", "Vile Parle"],
6     "Jogeshwari": ["Goregaon"],
7     "Vile Parle": ["Santacruz"],
8     "Goregaon": ["Ram Mandir"],
9 }
10
11 # BFS
12 def bfs(start, end):
13     queue = deque([start])
14     visited = set()
15     while queue:
16         node = queue.popleft()
17         if node == end:
18             return node
19         visited.add(node)
20         for neighbor in graph[node]:
21             if neighbor not in visited:
22                 queue.append(neighbor)
23
24 # Shortest Path
25 start = "Andheri"
26 end = "Dadar"
27 shortest_path = bfs(start, end)
28
29 # DFS Traversal
30 def dfs(node):
31     visited = set()
32     stack = [node]
33     while stack:
34         node = stack.pop()
35         if node not in visited:
36             visited.add(node)
37             print(node)
38             for neighbor in graph[node]:
39                 stack.append(neighbor)
40
41 # Path Found
42 path_found = bfs(start, end)
43
44 # BFS vs DFS Comparison
45
46 # Execution Time
47 start_time = time.time()
48 bfs(start, end)
49 end_time = time.time()
50 print("BFS Time: {}".format(end_time - start_time))
51
52 # DFS Time
53 start_time = time.time()
54 dfs(start)
55 end_time = time.time()
56 print("DFS Time: {}".format(end_time - start_time))
57
58 # Conclusion
59 # BFS gives the shortest path in an unweighted graph.
60 # DFS explores one branch completely before backtracking and may not always find the shortest path.
61
62 # C:\VACS 5380>
```

C.

```
indexpy 2 @ Relative.dh
1 from collections import deque
2 import time
3
4 graph = {
5     "Andheri": ["Jogeshwari", "Vile Parle"],
6     "Jogeshwari": ["Goregaon"],
7     "Vile Parle": ["Santacruz"],
8     "Goregaon": ["Ram Mandir"],
9 }
10
11 # BFS
12 def bfs(start, end):
13     queue = deque([start])
14     visited = set()
15     while queue:
16         node = queue.popleft()
17         if node == end:
18             return node
19         visited.add(node)
20         for neighbor in graph[node]:
21             if neighbor not in visited:
22                 queue.append(neighbor)
23
24 # Shortest Path
25 start = "Andheri"
26 end = "Dadar"
27 shortest_path = bfs(start, end)
28
29 # DFS Traversal
30 def dfs(node):
31     visited = set()
32     stack = [node]
33     while stack:
34         node = stack.pop()
35         if node not in visited:
36             visited.add(node)
37             print(node)
38             for neighbor in graph[node]:
39                 stack.append(neighbor)
40
41 # Path Found
42 path_found = bfs(start, end)
43
44 # BFS vs DFS Comparison
45
46 # Execution Time
47 start_time = time.time()
48 bfs(start, end)
49 end_time = time.time()
50 print("BFS Time: {}".format(end_time - start_time))
51
52 # DFS Time
53 start_time = time.time()
54 dfs(start)
55 end_time = time.time()
56 print("DFS Time: {}".format(end_time - start_time))
57
58 # Conclusion
59 # BFS gives the shortest path in an unweighted graph.
60 # DFS explores one branch completely before backtracking and may not always find the shortest path.
61
62 # C:\VACS 5380>
```

NAME – UNNATI RATHOD
ROLL NO – T109
SUBJECT – AI